```
  cello: "string",
  clarinet: "woodwind",
  drum: "percussion",
  oboe: "woodwind",
  trumpet: "brass",
  violin: "string"
}
puts "An oboe is a #{instrument_section[:oboe]} instrument"
```

produces:

```
An oboe is a woodwind instrument
```

This syntax was added, in part, to be familiar to programmers familiar with JavaScript and Python, both of which use a colon as a separator in key/value pairs.

Control Structures

Ruby has all the usual control structures, such as `if` statements and `while` loops. Java or JavaScript programmers may be surprised by the lack of braces around the bodies of these statements. Instead, Ruby uses the keyword `end` to signify the end of a body of a control structure:

intro/weekdays.rb

```
today = Time.now

if today.saturday?
  puts "Do chores around the house"
elsif today.sunday?
  puts "Relax"
else
  puts "Go to work"
end
```

produces:

```
Relax
```

One thing you might find unusual is that in the second clause Ruby uses the keyword `elsif`—one word, missing an "e"—to indicate "else if". Breaking that keyword up into `else if` would be a syntax error.

Similarly, `while` statements are terminated with `end` and loop as long as the condition on the line with the `while` is true:

```
num_pallets = 0
weight = 0
while weight < 100 && num_pallets <= 5
  pallet = next_pallet()
  weight += pallet.weight
  num_pallets += 1
end
```

Most lines that look like statements in Ruby are actually expressions that return a value, which means you can use those expressions as conditions. For example, the `Kernel` method `gets` returns the next line from the standard input stream or `nil` when the end of the file is reached. Because Ruby treats `nil` as a false value in conditions, you could write the following to process the lines in a file: